AI Study Assistant - Project Documentation

1. Introduction

The AI Study Assistant is an intelligent educational support system designed to help students learn more effectively using Artificial Intelligence. The application allows students to ask questions, receive explanations, summarize learning material, and understand complex concepts in a simplified way.

Traditional learning methods often require students to search through large amounts of information to understand a single concept. The AI Study Assistant solves this problem by providing instant AI-generated responses that are clear, structured, and easy to understand.

The main objective of this project is to create a smart assistant that improves the learning experience by combining Artificial Intelligence with modern web technologies. The system acts as a virtual tutor that students can interact with anytime.

The application integrates a frontend interface for user interaction and a backend server that communicates with an AI model through APIs. When a student asks a question, the system sends the request to the AI model and returns the generated answer in real time.

2. Problem Statement

Students often face several challenges while studying, such as difficulty understanding complex topics, lack of instant guidance when studying alone, and spending too much time searching multiple sources for explanations.

Most existing learning platforms provide static content rather than interactive explanations. Therefore, there is a need for a system that can provide instant answers, simplify complex concepts, assist students during self-study, and improve productivity.

The AI Study Assistant addresses these problems by acting as a digital tutor that students can interact with anytime to receive quick and helpful explanations.

3. Objectives of the Project

The primary objectives of the AI Study Assistant project are:

- Develop an AI-powered chatbot for educational support.
- Provide instant explanations for academic questions.

• Simplify complex concepts for students.
• Integrate AI APIs into a web-based platform.
• Create a simple and user-friendly interface for interaction.
• Demonstrate the practical use of Artificial Intelligence in education.

These objectives help create a system that enhances learning efficiency and supports independent study.

4. Scope of the Project

The scope of the AI Study Assistant includes the development of a web-based chatbot system that allows students to ask academic questions and receive AI-generated responses.

The system focuses on educational question answering and concept explanation. It demonstrates how Artificial Intelligence can assist students in real-time learning environments.

However, the current version does not include full course management systems, student progress tracking, or advanced analytics. These features may be included in future versions of the system.

5. System Architecture

The AI Study Assistant follows a layered architecture consisting of the frontend, backend, and AI service layer.

Frontend Layer:
The frontend handles user interaction and displays the chatbot interface. Technologies used include HTML, CSS, Tailwind CSS, and JavaScript.

Backend Layer:
The backend processes user queries and communicates with the AI model. It is developed using Python and Flask.

AI Service Layer:
This layer integrates the application with AI APIs that process user queries and generate intelligent responses.

This architecture ensures smooth communication between the user interface and the AI engine.

6. Technologies Used

Frontend Technologies:
HTML – Used to structure the web interface.

CSS – Used for styling and layout.
Tailwind CSS – Provides modern and responsive design components.
JavaScript – Handles interaction and API requests.

Backend Technologies:
Python – Main programming language for backend logic.
Flask – Lightweight web framework for building APIs.

AI Integration:
AI APIs are used to generate intelligent responses and explanations for student queries.

7. System Workflow

Step 1: The user enters a question in the chatbot interface.
Step 2: The frontend sends the query to the backend using an API request.
Step 3: The backend processes the request and sends the query to the AI API.
Step 4: The AI model analyzes the question and generates a response.
Step 5: The backend receives the AI response.
Step 6: The response is sent back to the frontend.
Step 7: The chatbot displays the answer to the user.

This workflow ensures quick and efficient interaction between the user and the AI system.

8. Features of the System

• AI-powered question answering
• Instant explanations for academic topics
• Interactive chatbot interface
• Real-time response generation
• Clean and user-friendly UI
• AI API integration for intelligent responses

These features make the system useful for students who want quick and clear explanations while studying.

9. Advantages of the System

• Provides instant learning support.
• Reduces time spent searching for information.
• Supports self-learning environments.

• Accessible anytime through the web interface.
• Simplifies complex topics for students.

These advantages make the AI Study Assistant a powerful educational support tool.

10. Limitations of the System

• AI responses may occasionally be inaccurate.
• The system requires an internet connection.
• No built-in student progress tracking.
• Limited personalization in the current version.

Future improvements can address these limitations.

11. Future Enhancements

Future versions of the AI Study Assistant may include:

• Voice-based interaction
• Mobile application development
• Personalized learning recommendations
• Document and PDF summarization
• Multi-language support
• Integration with learning management systems

These enhancements would significantly expand the capabilities of the system.

12. Conclusion

The AI Study Assistant demonstrates how Artificial Intelligence can enhance the learning process by providing instant explanations and academic assistance.

By combining web technologies with AI APIs, the system creates an intelligent platform that helps students understand complex topics quickly. Although the current system focuses on core functionality, it has strong potential for expansion with advanced AI features and educational tools.

This project highlights the growing importance of AI-powered learning systems in modern education.

```python
from flask import Flask, request, jsonify, render_template
```

```python
from google import genai
import time

app = Flask(__name__)

# AI client
API_KEY = "AIzaSyCuegXyceRlXpztzGpuO5LNmYVyPNNdxsM"
client = genai.Client(api_key=API_KEY)

def ask_ai(question):
    try:
        response = client.models.generate_content(
            model="gemini-2.5-flash",
            contents=[question]
        )
        return response.text.strip()
    except Exception as e:
        return f"AI Error: {str(e)}"

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/ask", methods=["POST"])
def ask():
    try:
        data = request.get_json()

        if not data:
            return jsonify({"answer": "No data received", "time": "0.00"})

        question = data.get("question","")

        if question == "":
            return jsonify({"answer": "Please enter a question", "time": "0.00"})

        start = time.time()

        answer = ask_ai(question)

        end = time.time()

        return jsonify({
```

```
            "answer": answer,
            "time": f"{end-start:.2f}"
        })

    except Exception as e:
        return jsonify({
            "answer": f"Server error: {str(e)}",
            "time": "0.00"
        })

if __name__ == "__main__":
    app.run(debug=True)
```

backed

frontend

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AI Study Assistant</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <style>
    .fade-in { animation: fadeIn 0.3s ease forwards; }
    @keyframes fadeIn { 0% {opacity:0; transform:translateY(10px);} 100%
{opacity:1; transform:translateY(0);} }
    .typing-dot { display:inline-block; width:8px; height:8px; margin:0 2px;
background-color:#888; border-radius:50%; animation: blink 1.4s infinite both; }
    .typing-dot:nth-child(1) {animation-delay:0s;}
    .typing-dot:nth-child(2) {animation-delay:0.2s;}
    .typing-dot:nth-child(3) {animation-delay:0.4s;}
    @keyframes blink { 0%,80%,100% {opacity:0;} 40% {opacity:1;} }
  </style>
</head>
<body class="bg-gray-100 dark:bg-gray-900 flex flex-col items-center justify-
center min-h-screen transition-colors duration-300">

<div class="w-full max-w-md bg-white dark:bg-gray-800 shadow-xl rounded-xl p-5
flex flex-col">
  <div class="flex justify-between items-center mb-4">
    <h1 class="text-2xl font-bold text-gray-900 dark:text-gray-100">📚 AI Study
Assistant</h1>
```

```
    <button id="themeToggle" class="text-gray-600 dark:text-gray-300
focus:outline-none">🌓</button>
  </div>

  <div id="chat" class="flex flex-col gap-3 mb-4 overflow-y-auto h-96 p-3 border
rounded-lg border-gray-300 dark:border-gray-700 bg-gray-50 dark:bg-gray-
900"></div>

  <div class="flex gap-2">
    <input id="question" type="text" placeholder="Type your question..."
class="flex-1 p-3 border rounded-lg border-gray-300 dark:border-gray-600
focus:outline-none focus:ring-2 focus:ring-blue-400 dark:bg-gray-700 dark:text-
gray-100">
    <button id="sendBtn" class="bg-blue-500 text-white px-5 py-3 rounded-lg
hover:bg-blue-600 transition-all">Send</button>
  </div>
</div>

<script>
const chat = document.getElementById('chat');
const input = document.getElementById('question');
const sendBtn = document.getElementById('sendBtn');
const themeToggle = document.getElementById('themeToggle');
let darkMode = false;

// Toggle theme
themeToggle.addEventListener('click', () => {
  darkMode = !darkMode;
  document.body.classList.toggle('dark', darkMode);
});

// Add chat message
function addMessage(text, sender, isTyping=false){
  const msg = document.createElement('div');
  msg.className = `fade-in ${sender==='user'?'self-end bg-blue-500 text-
white':'self-start bg-gray-200 dark:bg-gray-700 text-gray-800 dark:text-gray-
100'} p-3 rounded-xl max-w-[80%] break-words`;
  if(isTyping){
    msg.innerHTML = '<span class="typing-dot"></span><span class="typing-
dot"></span><span class="typing-dot"></span>';
  } else msg.innerText = text;
  chat.appendChild(msg);
  chat.scrollTop = chat.scrollHeight;
  return msg;
}
```

```
// Send question
async function sendQuestion(){
  const question = input.value.trim();
  if(!question) return;
  addMessage(question,'user');
  input.value='';
  const typingMsg = addMessage('', 'ai', true);

  try{
    const res = await fetch('/ask',{
      method:'POST',
      headers:{'Content-Type':'application/json'},
      body: JSON.stringify({question})
    });
    const data = await res.json().catch(()=>({answer:"⚠ Invalid server
response", time:"0.00"}));
    typingMsg.innerText = `${data.answer}\n⏱ ${data.time}`;
  } catch(err){
    typingMsg.innerText = `⚠ Error: ${err}`;
  }
}

sendBtn.addEventListener('click', sendQuestion);
input.addEventListener('keydown', e => {if(e.key==='Enter') sendQuestion();});
input.focus();
</script>
</body>
</html>
```

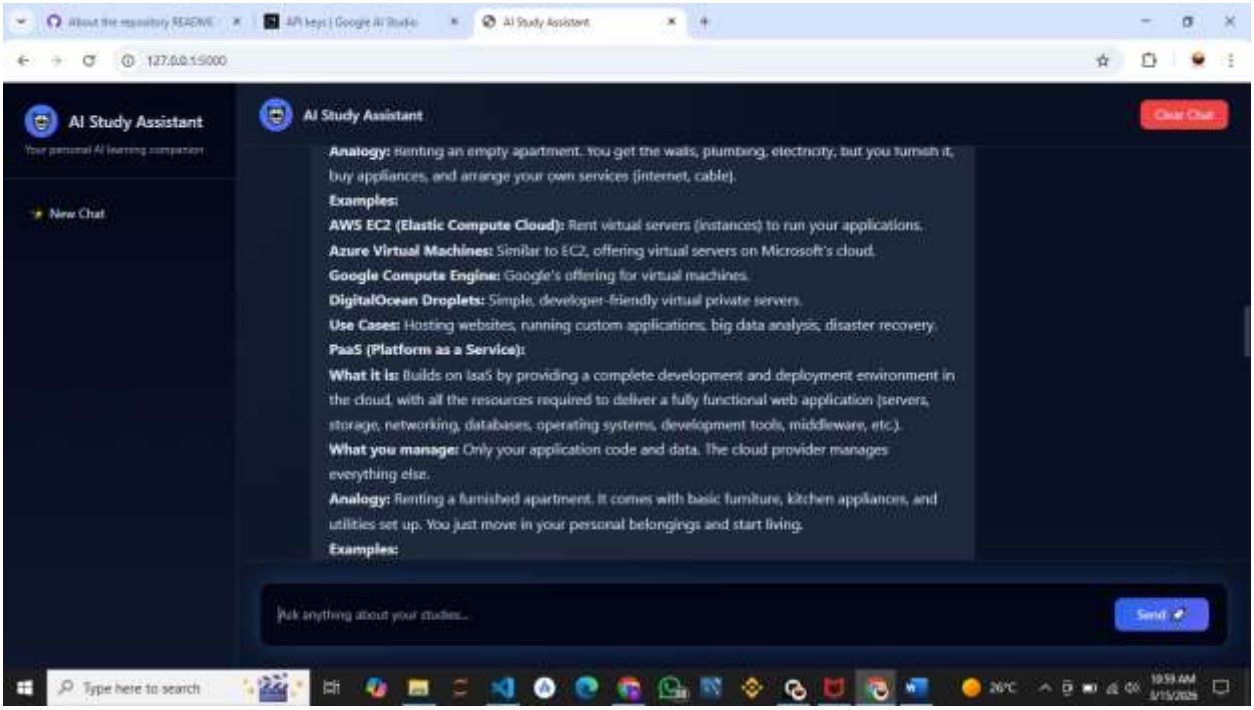
http://127.0.0.1:5000

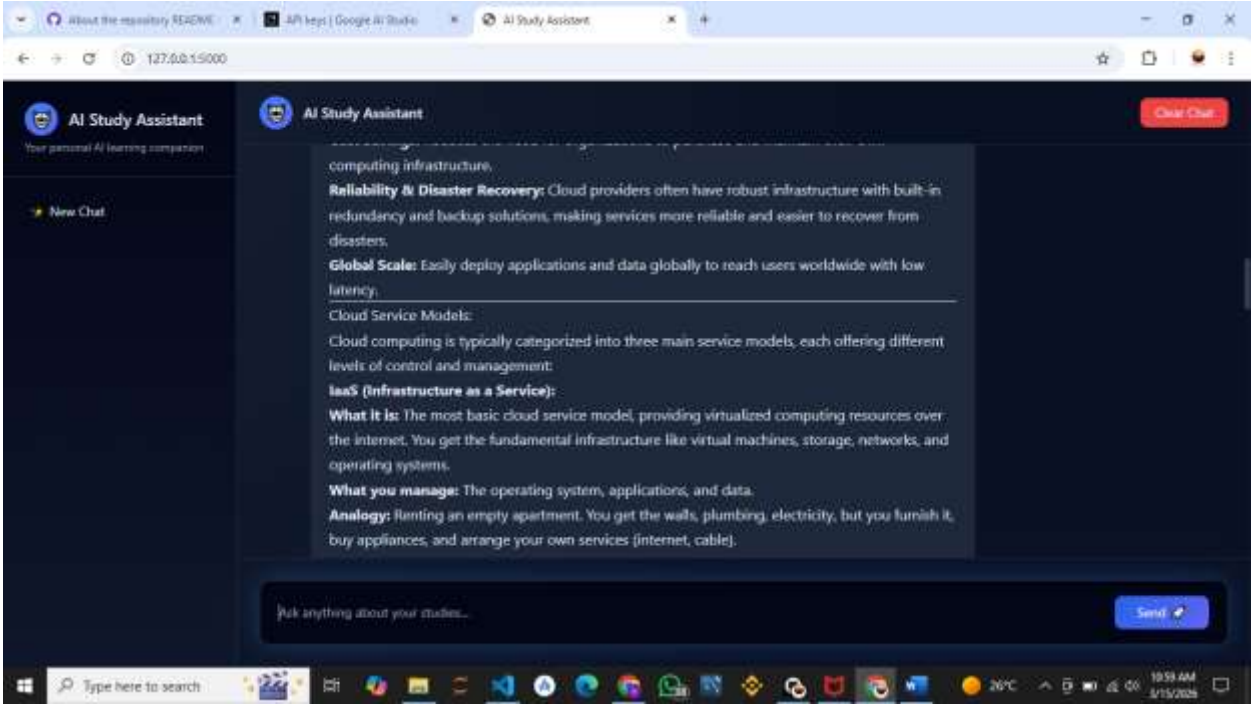
http://127.0.0.1:5000

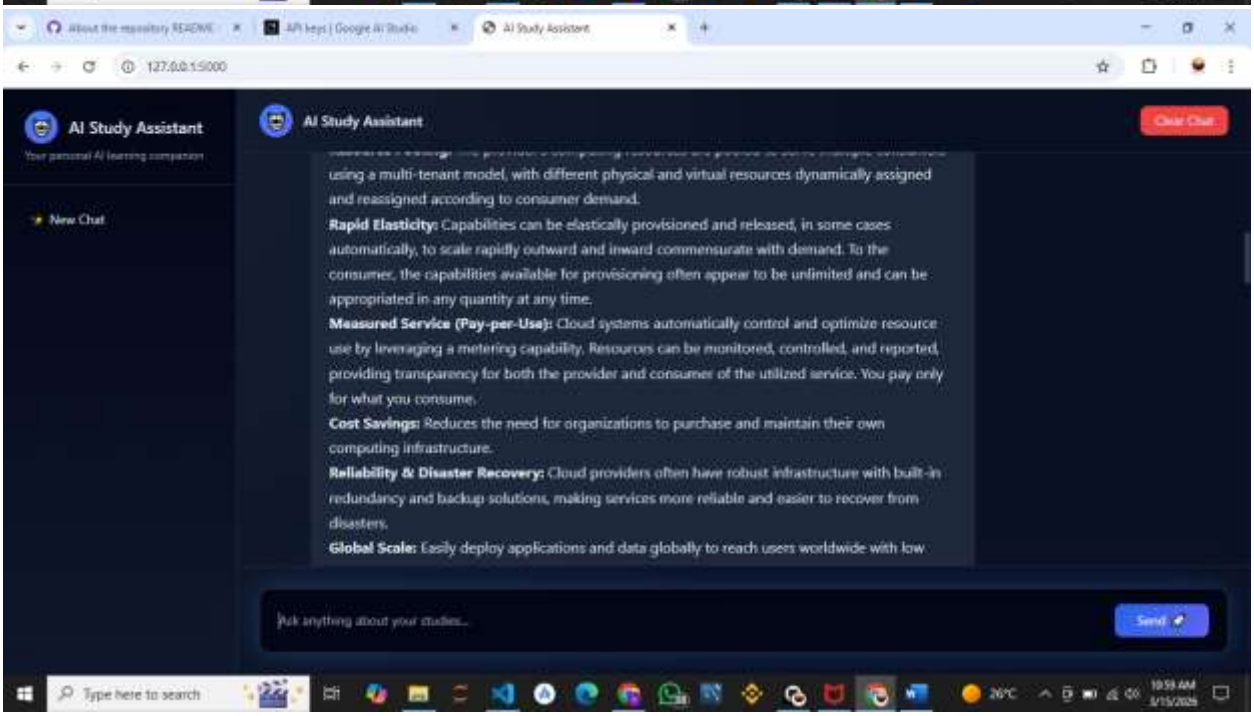
```python
from flask import Flask, request, jsonify, render_template

from google import genai
import time
import os
import logging

app = Flask(__name__)

# Logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)

# API Key (env variable supported)
API_KEY = os.getenv("GEMINI_API_KEY", "AIzaSyCQXBaxgwif4PEfKycMFAk_ywAZDWfBwHs")

# AI client
client = genai.Client(api_key=API_KEY)


# AI function (LOGIC SAME)
def ask_ai(question):
    try:
        response = client.models.generate_content(
            model="gemini-2.5-flash",
            contents=[question]
        )

        if hasattr(response, "text") and response.text:
            return response.text.strip()

        return "AI returned empty response."

    except Exception as e:
        logging.error(f"AI Error: {e}")
        return f"AI Error: {str(e)}"


@app.route("/")
def home():
    return render_template("index.html")
```

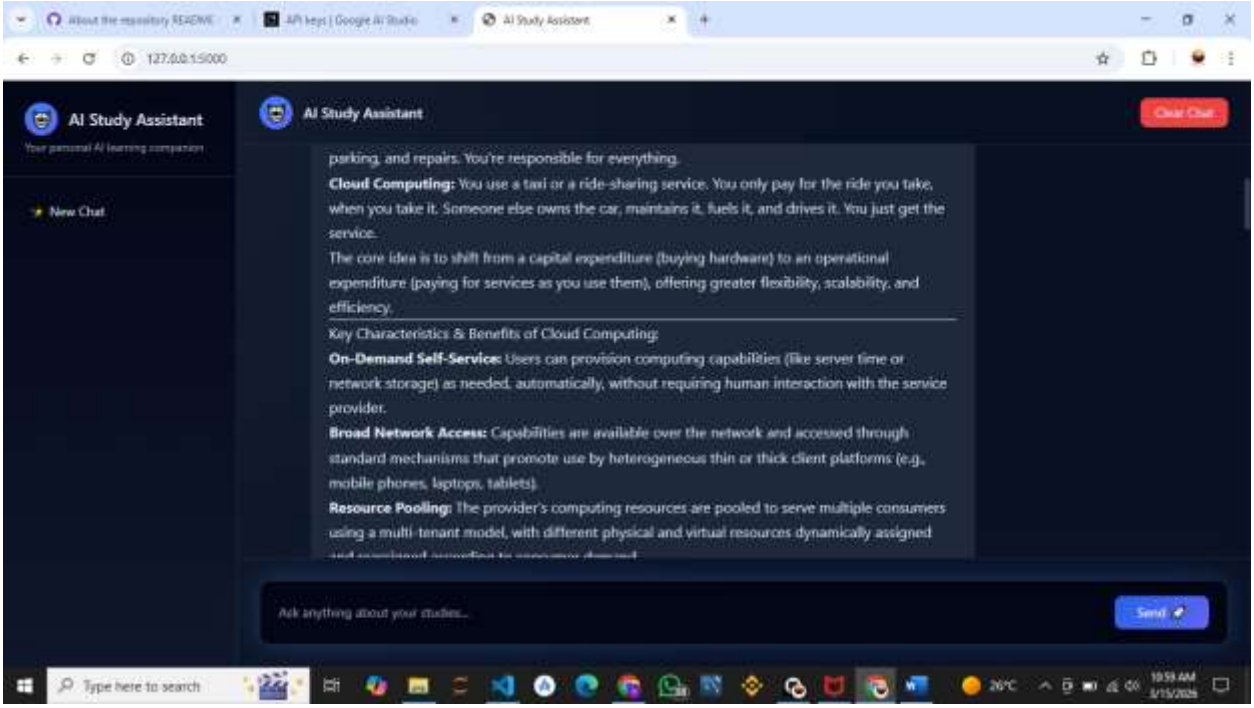
```python
# ASK API (LOGIC SAME)
@app.route("/ask", methods=["POST"])
def ask():
    try:
        data = request.get_json()

        if not data:
            return jsonify({"answer": "No data received", "time": "0.00"})

        question = data.get("question", "")

        if question == "":
            return jsonify({"answer": "Please enter a question", "time": "0.00"})

        start = time.perf_counter()

        answer = ask_ai(question)

        end = time.perf_counter()

        return jsonify({
            "answer": answer,
            "time": f"{end-start:.2f}"
        })

    except Exception as e:
        logging.error(e)
        return jsonify({
            "answer": f"Server error: {str(e)}",
            "time": "0.00"
        })


if __name__ == "__main__":
    app.run(debug=True)
```

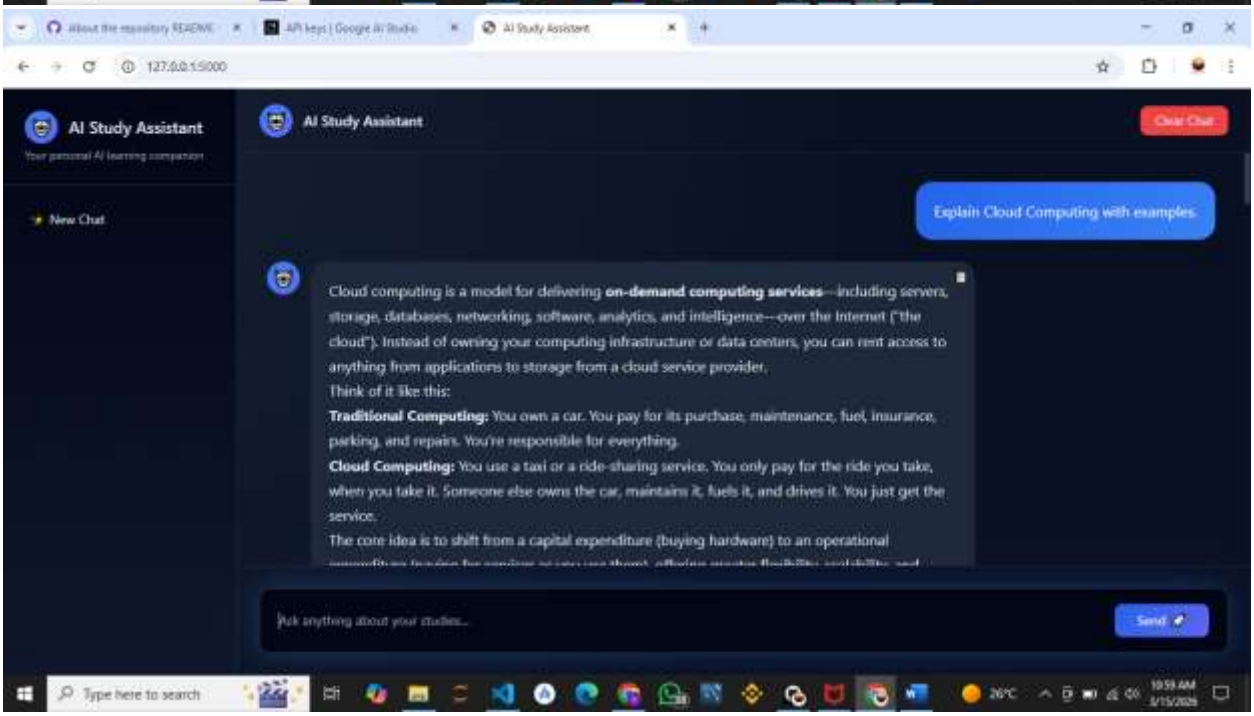
About the repository README
API keys | Google AI Studio
AI Study Assistant
127.0.0.1:5000
AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
utilities set up. You just move in your personal belongings and start living.
Examples:
AWS Elastic Beanstalk: Deploy and scale web applications and services developed with popular languages like Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker.
Heroku: A popular platform for deploying, managing, and scaling modern apps.
Google App Engine: Develop and host web applications in Google's managed data centers.
Azure App Service: Build, deploy, and scale web apps and APIs.
Use Cases: Web application development and deployment, API hosting, rapid development environments.
SaaS (Software as a Service):
What it is: The most complete cloud service model, where the cloud provider hosts and manages the entire application, and you simply access it over the internet, usually through a web browser or a dedicated app.
What you manage: Nothing related to the infrastructure or software; you just use the application.
Analogy: Renting a hotel room. Everything is provided and managed for you—the room, the
Ask anything about your studies...
Send
Type here to search
26°C
10:59 AM
5/15/2026

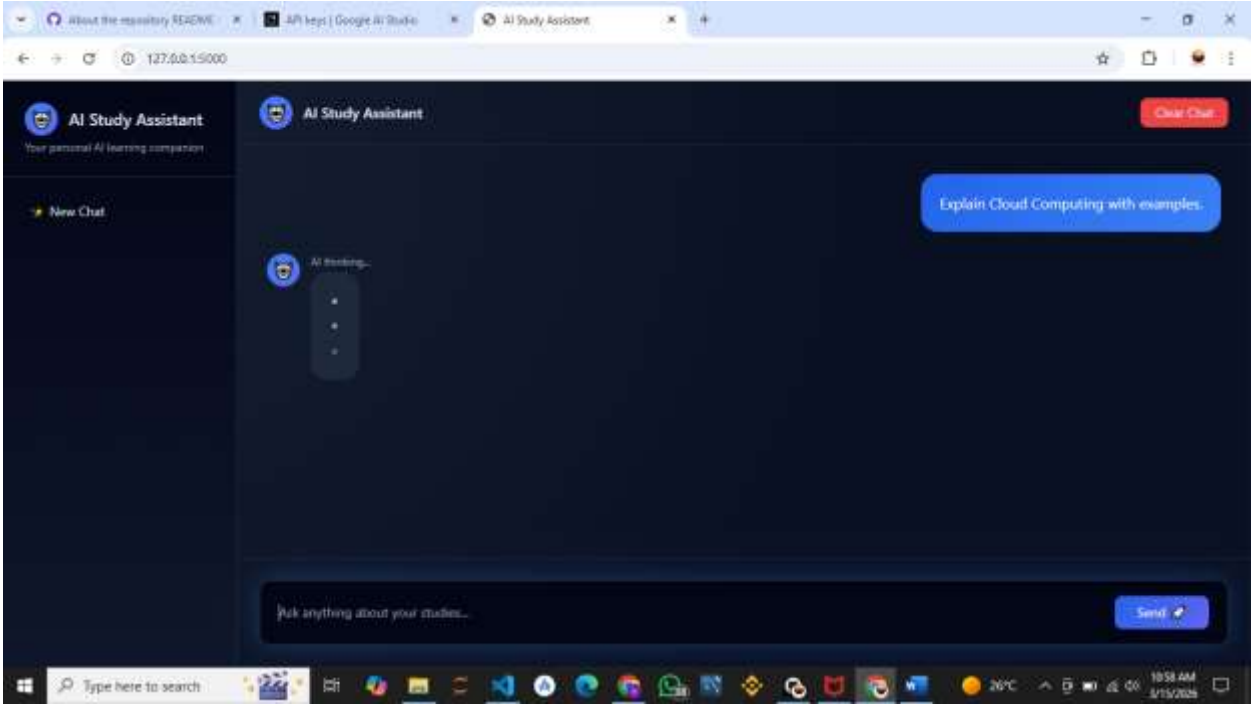

About the repository README
API keys | Google AI Studio
AI Study Assistant
127.0.0.1:5000
AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
Analogy: Renting an empty apartment. You get the walls, plumbing, electricity, but you furnish it, buy appliances, and arrange your own services (internet, cable).
Examples:
AWS EC2 (Elastic Compute Cloud): Rent virtual servers (instances) to run your applications.
Azure Virtual Machines: Similar to EC2, offering virtual servers on Microsoft's cloud.
Google Compute Engine: Google's offering for virtual machines.
DigitalOcean Droplets: Simple, developer-friendly virtual private servers.
Use Cases: Hosting websites, running custom applications, big data analysis, disaster recovery.
PaaS (Platform as a Service):
What it is: Builds on IaaS by providing a complete development and deployment environment in the cloud, with all the resources required to deliver a fully functional web application (servers, storage, networking, databases, operating systems, development tools, middleware, etc.).
What you manage: Only your application code and data. The cloud provider manages everything else.
Analogy: Renting a furnished apartment. It comes with basic furniture, kitchen appliances, and utilities set up. You just move in your personal belongings and start living.
Examples:
Ask anything about your studies...
Send
Type here to search
26°C
10:59 AM
5/15/2026

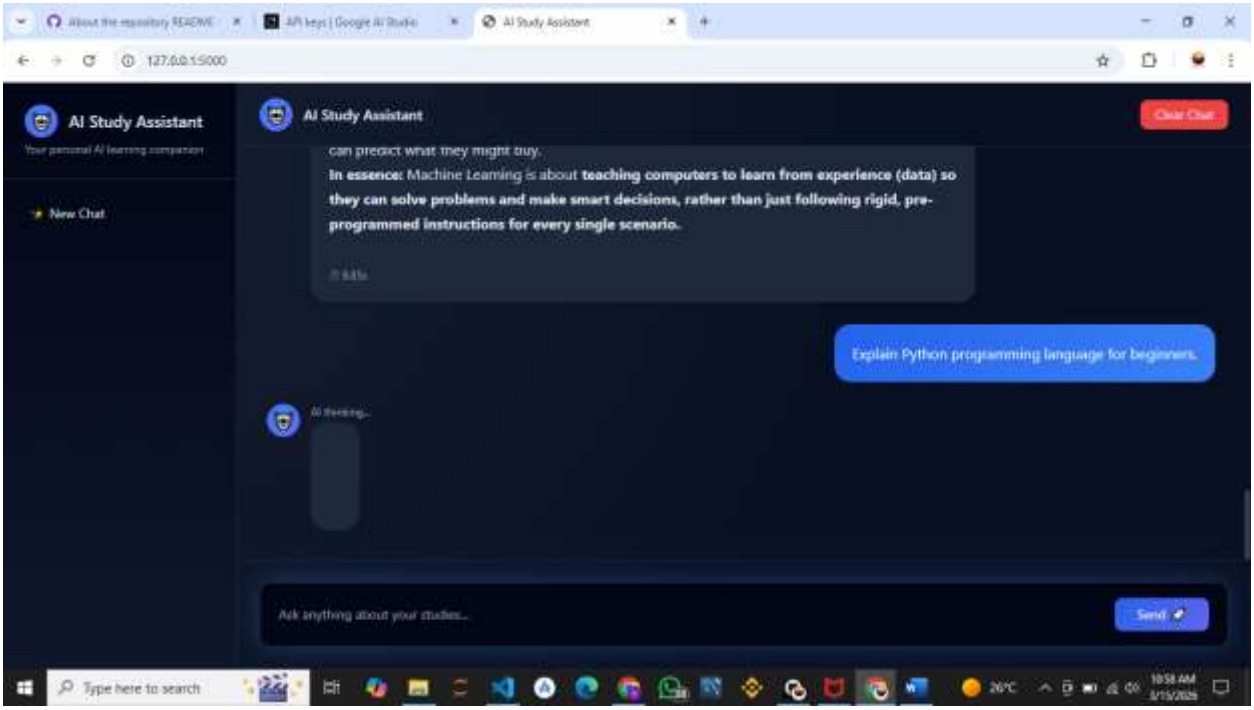

AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
computing infrastructure.
Reliability & Disaster Recovery: Cloud providers often have robust infrastructure with built-in redundancy and backup solutions, making services more reliable and easier to recover from disasters.
Global Scale: Easily deploy applications and data globally to reach users worldwide with low latency.
Cloud Service Models:
Cloud computing is typically categorized into three main service models, each offering different levels of control and management:
IaaS (Infrastructure as a Service):
What it is: The most basic cloud service model, providing virtualized computing resources over the internet. You get the fundamental infrastructure like virtual machines, storage, networks, and operating systems.
What you manage: The operating system, applications, and data.
Analogy: Renting an empty apartment. You get the walls, plumbing, electricity, but you furnish it, buy appliances, and arrange your own services (internet, cable).
Ask anything about your studies...
Send

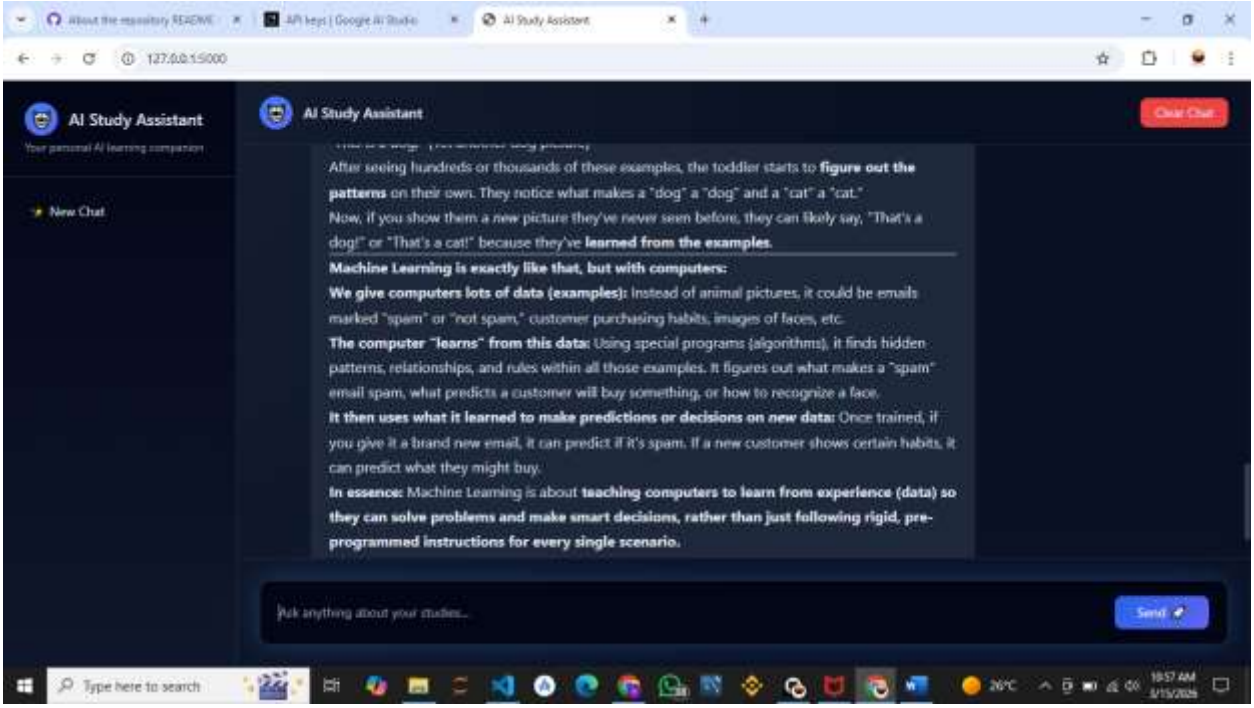

AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand.
Rapid Elasticity: Capabilities can be elastically provisioned and released, in some cases automatically, to scale rapidly outward and inward commensurate with demand. To the consumer, the capabilities available for provisioning often appear to be unlimited and can be appropriated in any quantity at any time.
Measured Service (Pay-per-Use): Cloud systems automatically control and optimize resource use by leveraging a metering capability. Resources can be monitored, controlled, and reported, providing transparency for both the provider and consumer of the utilized service. You pay only for what you consume.
Cost Savings: Reduces the need for organizations to purchase and maintain their own computing infrastructure.
Reliability & Disaster Recovery: Cloud providers often have robust infrastructure with built-in redundancy and backup solutions, making services more reliable and easier to recover from disasters.
Global Scale: Easily deploy applications and data globally to reach users worldwide with low
Ask anything about your studies...
Send

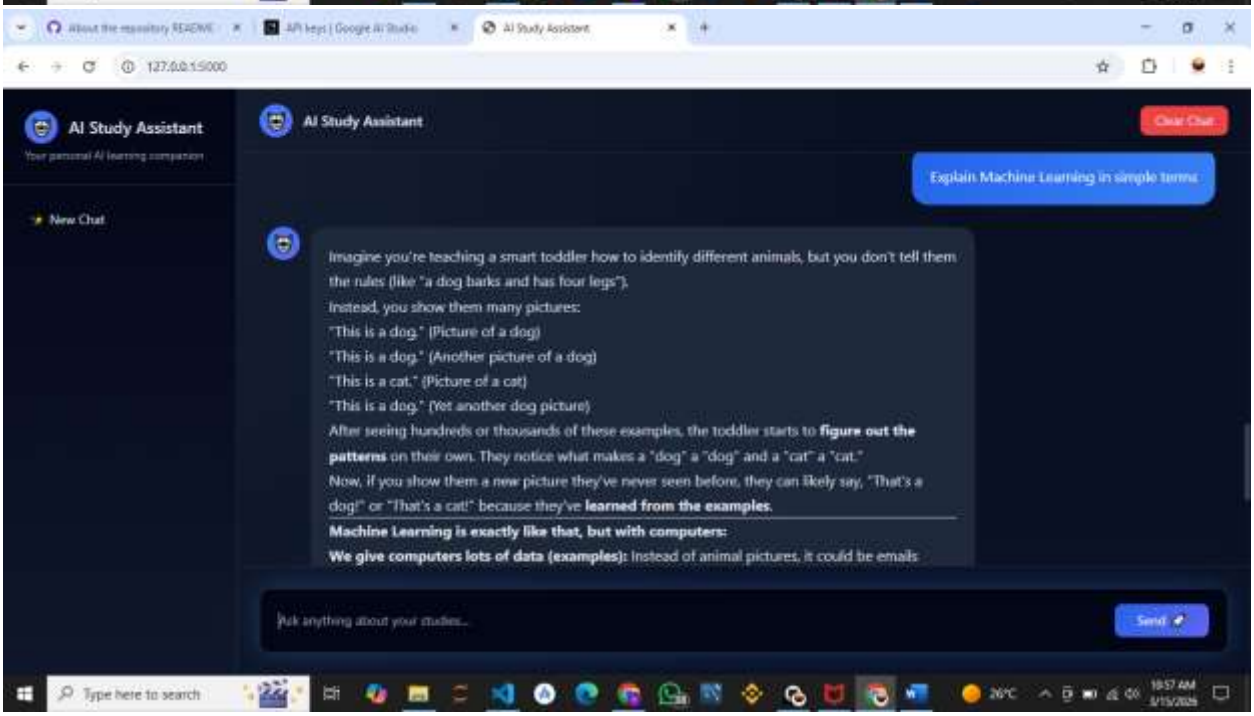

AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
parking, and repairs. You're responsible for everything.
Cloud Computing: You use a taxi or a ride-sharing service. You only pay for the ride you take, when you take it. Someone else owns the car, maintains it, fuels it, and drives it. You just get the service.
The core idea is to shift from a capital expenditure (buying hardware) to an operational expenditure (paying for services as you use them), offering greater flexibility, scalability, and efficiency.
Key Characteristics & Benefits of Cloud Computing:
On-Demand Self-Service: Users can provision computing capabilities (like server time or network storage) as needed, automatically, without requiring human interaction with the service provider.
Broad Network Access: Capabilities are available over the network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, laptops, tablets).
Resource Pooling: The provider's computing resources are pooled to serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned
Ask anything about your studies...
Send
Type here to search

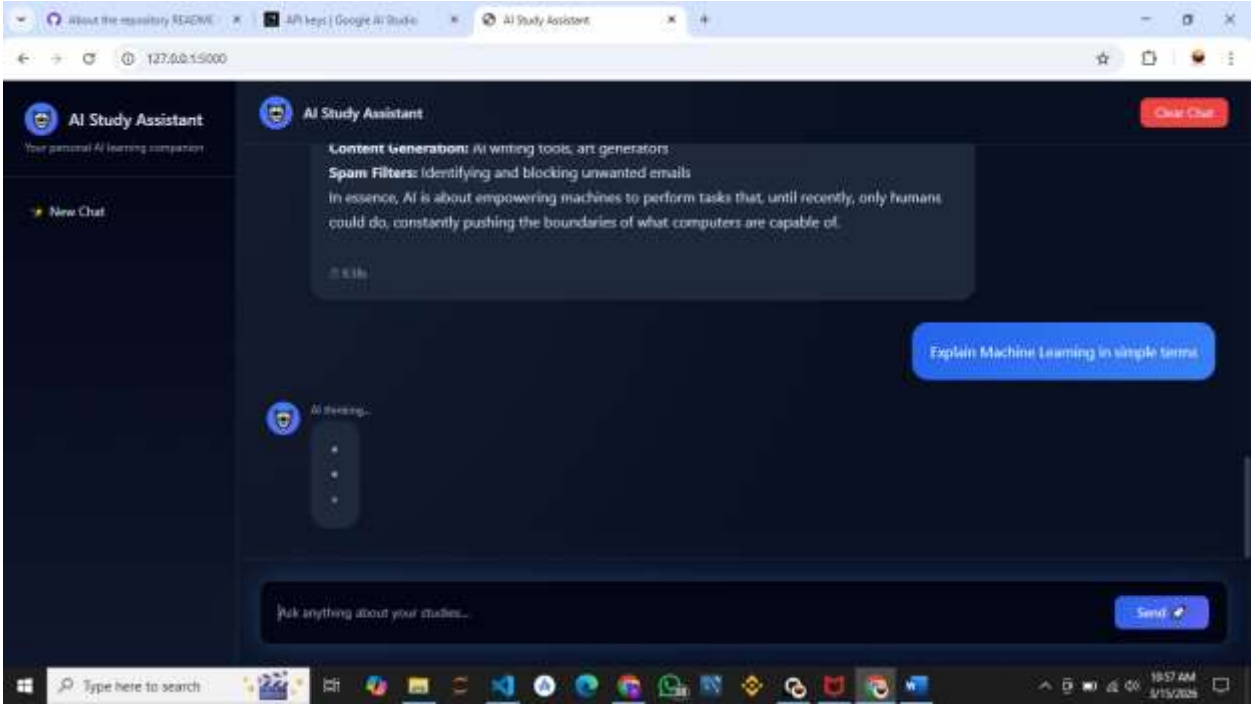

AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
Explain Cloud Computing with examples.
Cloud computing is a model for delivering on-demand computing services—including servers, storage, databases, networking, software, analytics, and intelligence—over the Internet ("the cloud"). Instead of owning your computing infrastructure or data centers, you can rent access to anything from applications to storage from a cloud service provider.
Think of it like this:
Traditional Computing: You own a car. You pay for its purchase, maintenance, fuel, insurance, parking, and repairs. You're responsible for everything.
Cloud Computing: You use a taxi or a ride-sharing service. You only pay for the ride you take, when you take it. Someone else owns the car, maintains it, fuels it, and drives it. You just get the service.
The core idea is to shift from a capital expenditure (buying hardware) to an operational
Ask anything about your studies...
Send
Type here to search

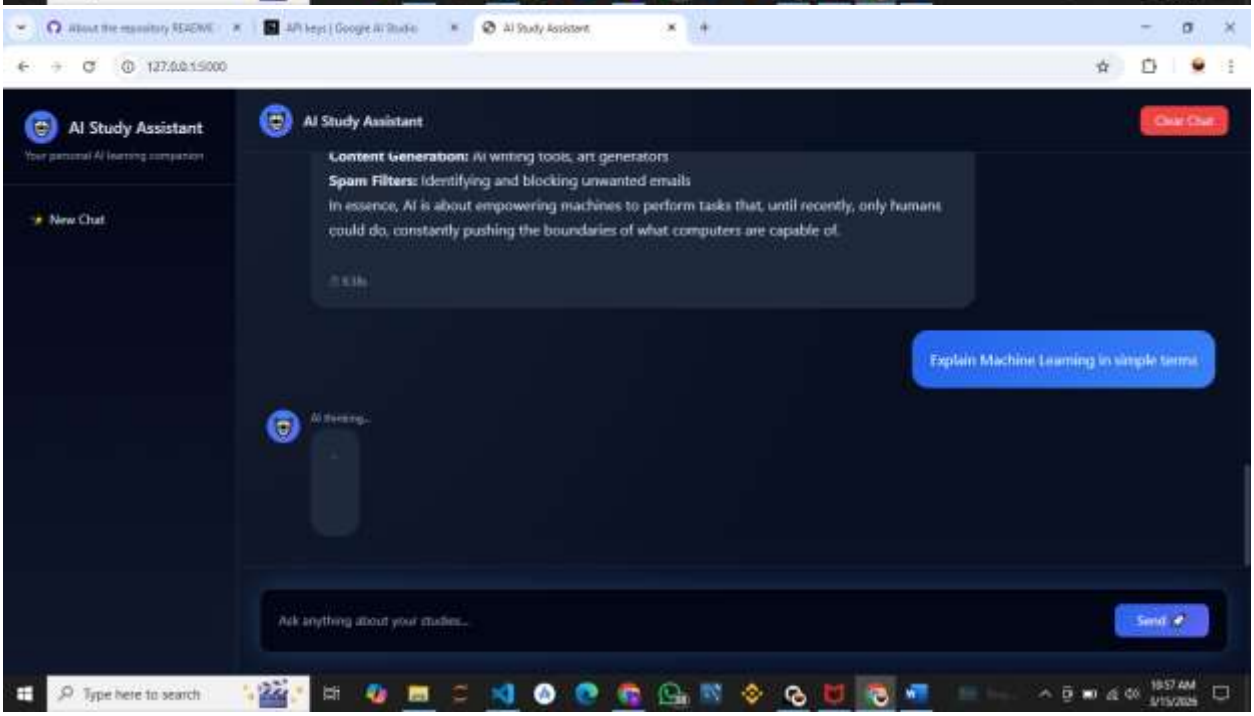

AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
Explain Cloud Computing with examples.
AI thinking...
Ask anything about your studies...
Send

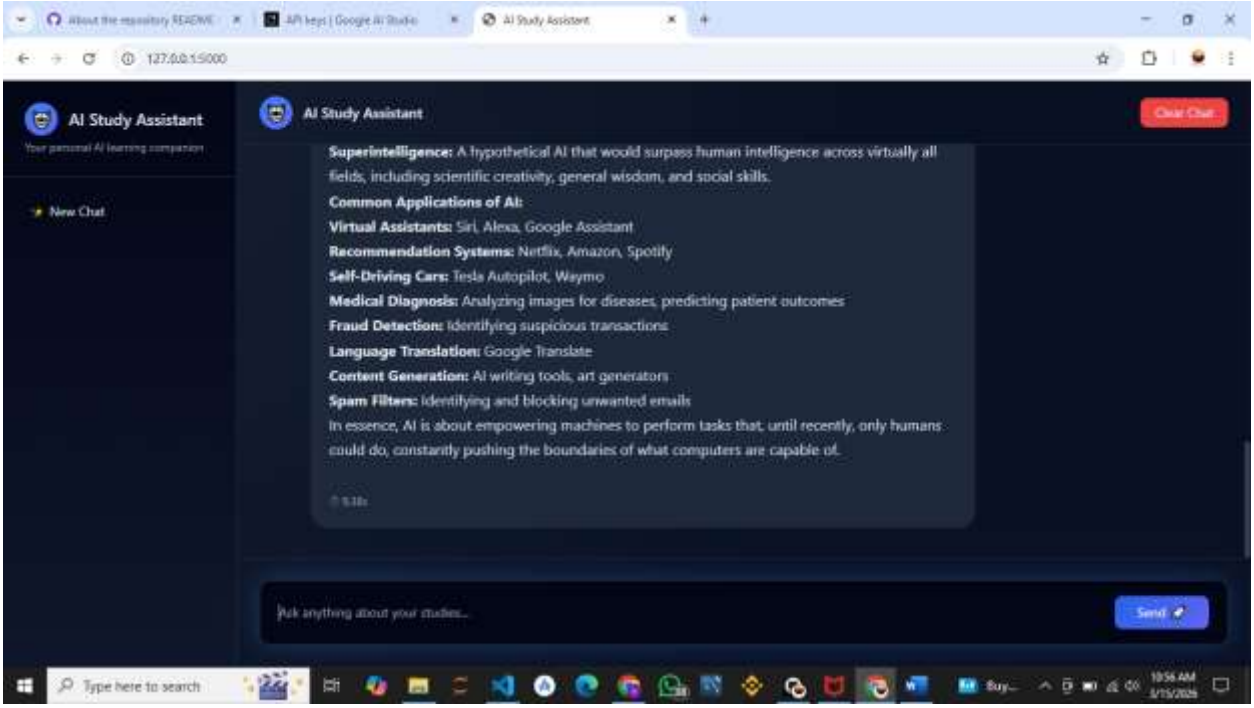

AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
can predict what they might buy.
In essence: Machine Learning is about teaching computers to learn from experience (data) so they can solve problems and make smart decisions, rather than just following rigid, pre-programmed instructions for every single scenario.
Explain Python programming language for beginners.
AI thinking...
Ask anything about your studies...
Send

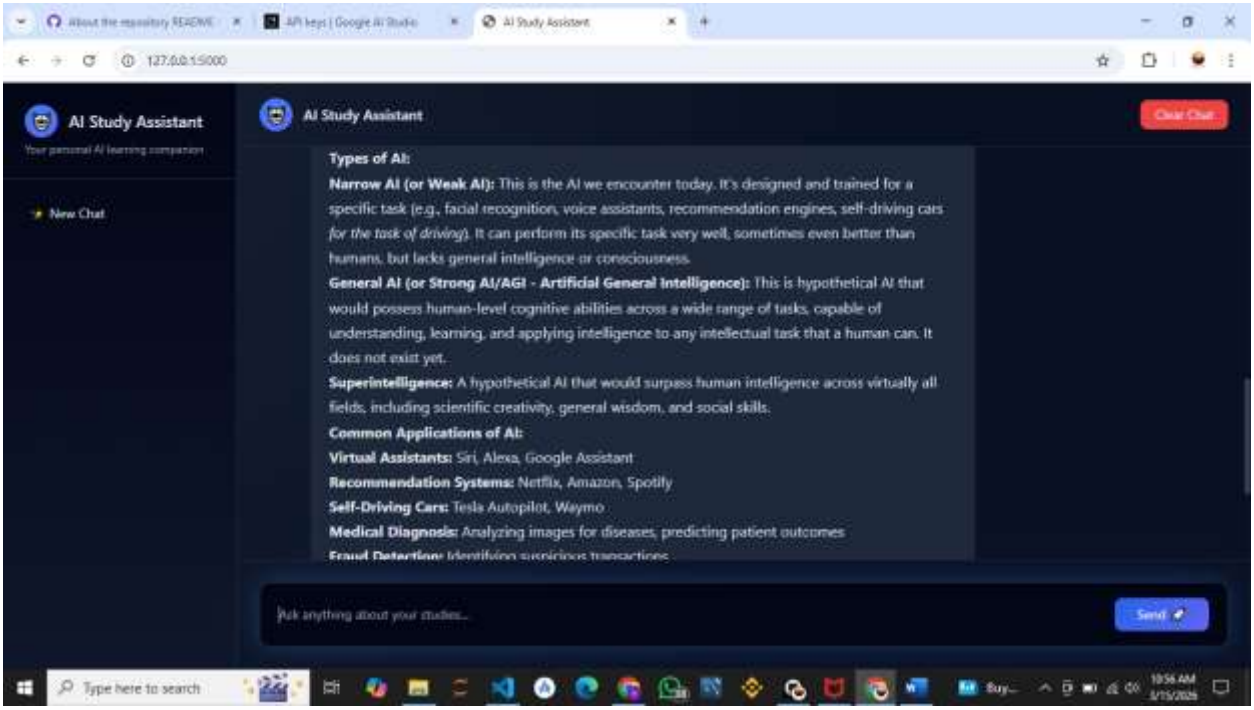

About the repository README
API keys | Google AI Studio
AI Study Assistant
127.0.0.1:5000
AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
After seeing hundreds or thousands of these examples, the toddler starts to **figure out the patterns** on their own. They notice what makes a "dog" a "dog" and a "cat" a "cat."
Now, if you show them a new picture they've never seen before, they can likely say, "That's a dog!" or "That's a cat!" because they've **learned from the examples**.
Machine Learning is exactly like that, but with computers:
We give computers lots of data (examples): Instead of animal pictures, it could be emails marked "spam" or "not spam," customer purchasing habits, images of faces, etc.
The computer "learns" from this data: Using special programs (algorithms), it finds hidden patterns, relationships, and rules within all those examples. It figures out what makes a "spam" email spam, what predicts a customer will buy something, or how to recognize a face.
It then uses what it learned to make predictions or decisions on new data: Once trained, if you give it a brand new email, it can predict if it's spam. If a new customer shows certain habits, it can predict what they might buy.
In essence: Machine Learning is about **teaching computers to learn from experience (data) so they can solve problems and make smart decisions, rather than just following rigid, pre-programmed instructions for every single scenario.**
Ask anything about your studies...
Send
Type here to search
26°C
19:57 AM
5/15/2026

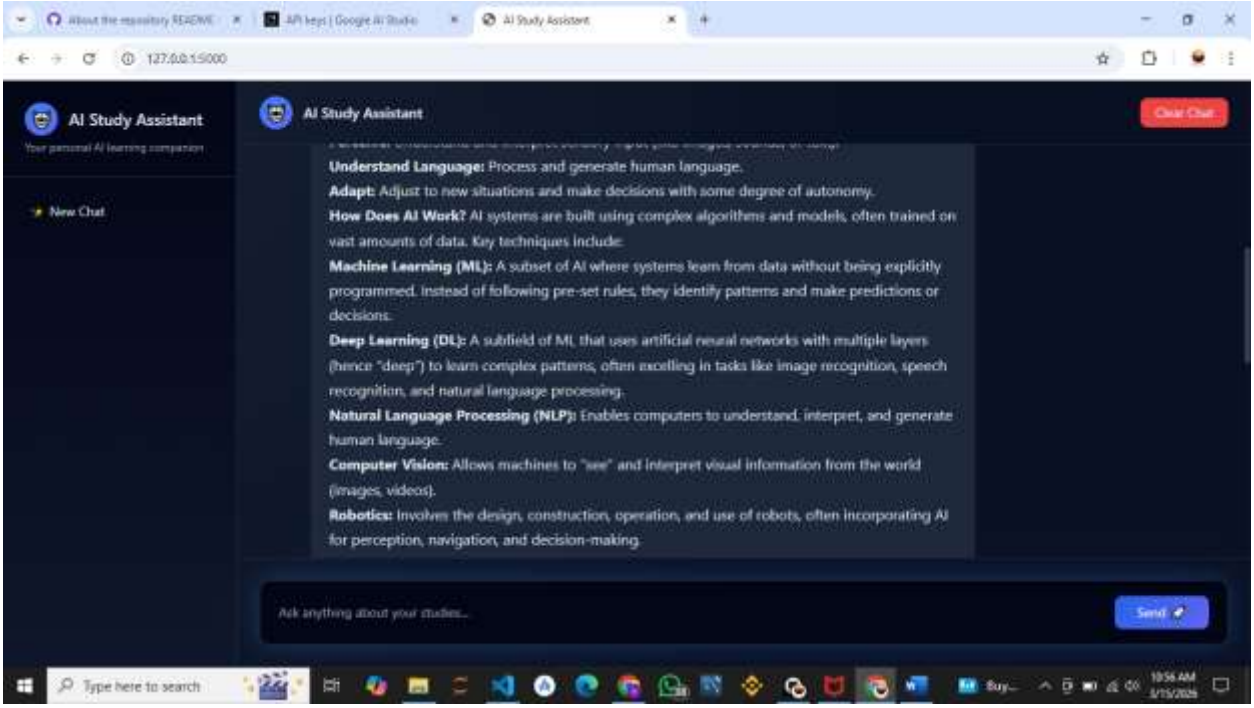

About the repository README
API keys | Google AI Studio
AI Study Assistant
127.0.0.1:5000
AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
Explain Machine Learning in simple terms
Imagine you're teaching a smart toddler how to identify different animals, but you don't tell them the rules (like "a dog barks and has four legs").
Instead, you show them many pictures:
"This is a dog." (Picture of a dog)
"This is a dog." (Another picture of a dog)
"This is a cat." (Picture of a cat)
"This is a dog." (Yet another dog picture)
After seeing hundreds or thousands of these examples, the toddler starts to **figure out the patterns** on their own. They notice what makes a "dog" a "dog" and a "cat" a "cat."
Now, if you show them a new picture they've never seen before, they can likely say, "That's a dog!" or "That's a cat!" because they've **learned from the examples**.
Machine Learning is exactly like that, but with computers:
We give computers lots of data (examples): Instead of animal pictures, it could be emails
Ask anything about your studies...
Send
Type here to search
26°C
19:57 AM
5/15/2026

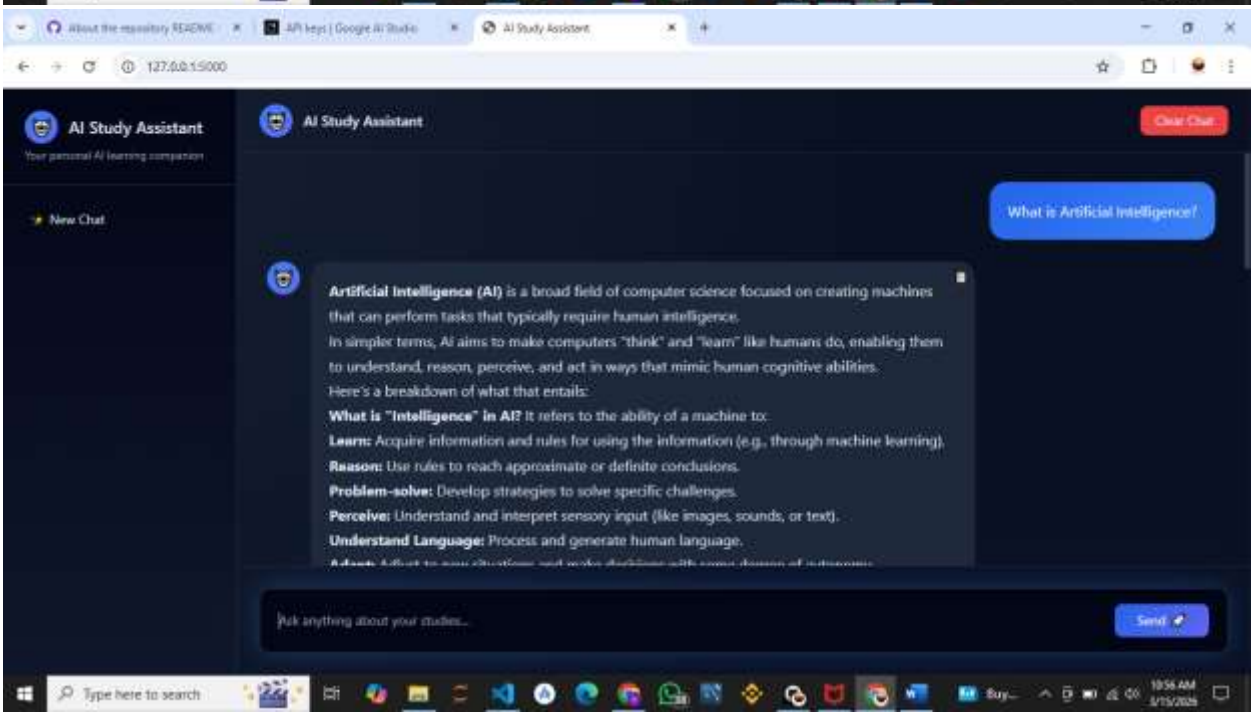

About the repository README
API keys | Google AI Studio
AI Study Assistant
127.0.0.1:5000
AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
Content Generation: AI writing tools, art generators
Spam Filters: Identifying and blocking unwanted emails
In essence, AI is about empowering machines to perform tasks that, until recently, only humans could do, constantly pushing the boundaries of what computers are capable of.
Explain Machine Learning in simple terms
AI thinking...
Ask anything about your studies...
Send
Type here to search

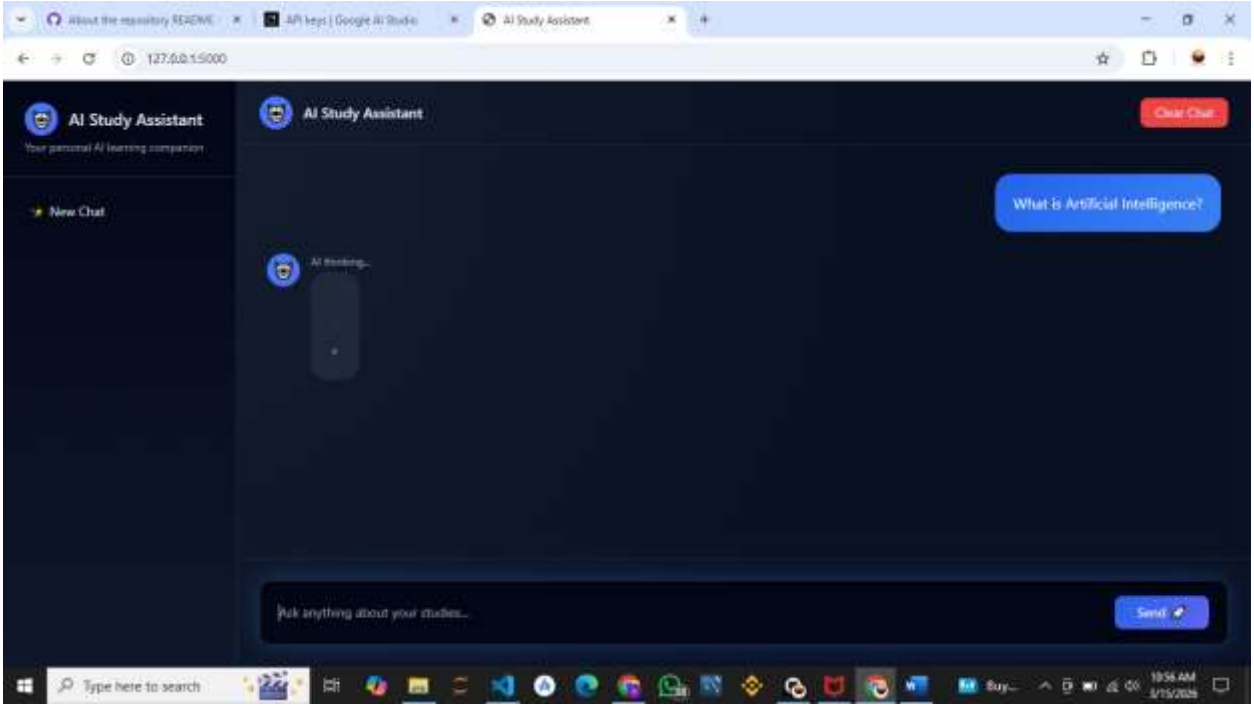

About the repository README
API keys | Google AI Studio
AI Study Assistant
127.0.0.1:5000
AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
Content Generation: AI writing tools, art generators
Spam Filters: Identifying and blocking unwanted emails
In essence, AI is about empowering machines to perform tasks that, until recently, only humans could do, constantly pushing the boundaries of what computers are capable of.
Explain Machine Learning in simple terms
AI thinking...
Ask anything about your studies...
Send
Type here to search

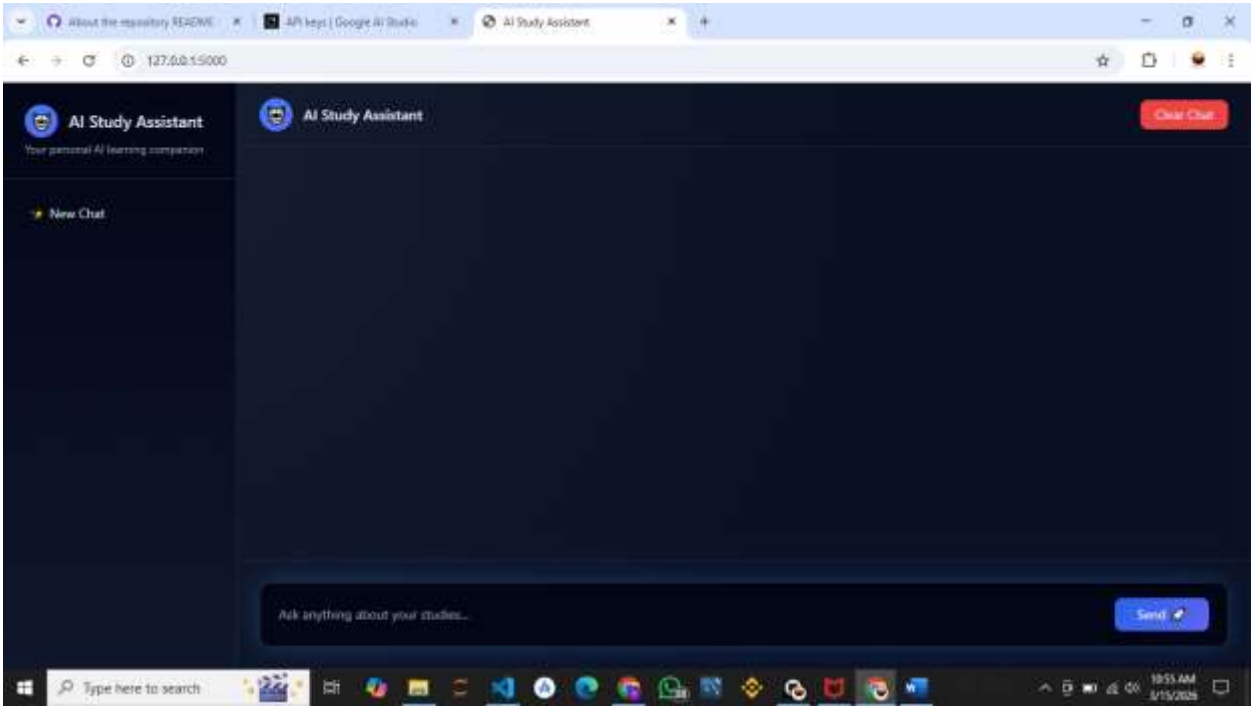

About the repository README
API keys | Google AI Studio
AI Study Assistant
127.0.0.1:5000
AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
Superintelligence: A hypothetical AI that would surpass human intelligence across virtually all fields, including scientific creativity, general wisdom, and social skills.
Common Applications of AI:
Virtual Assistants: Siri, Alexa, Google Assistant
Recommendation Systems: Netflix, Amazon, Spotify
Self-Driving Cars: Tesla Autopilot, Waymo
Medical Diagnosis: Analyzing images for diseases, predicting patient outcomes
Fraud Detection: Identifying suspicious transactions
Language Translation: Google Translate
Content Generation: AI writing tools, art generators
Spam Filters: Identifying and blocking unwanted emails
In essence, AI is about empowering machines to perform tasks that, until recently, only humans could do, constantly pushing the boundaries of what computers are capable of.
Ask anything about your studies...
Send
Type here to search

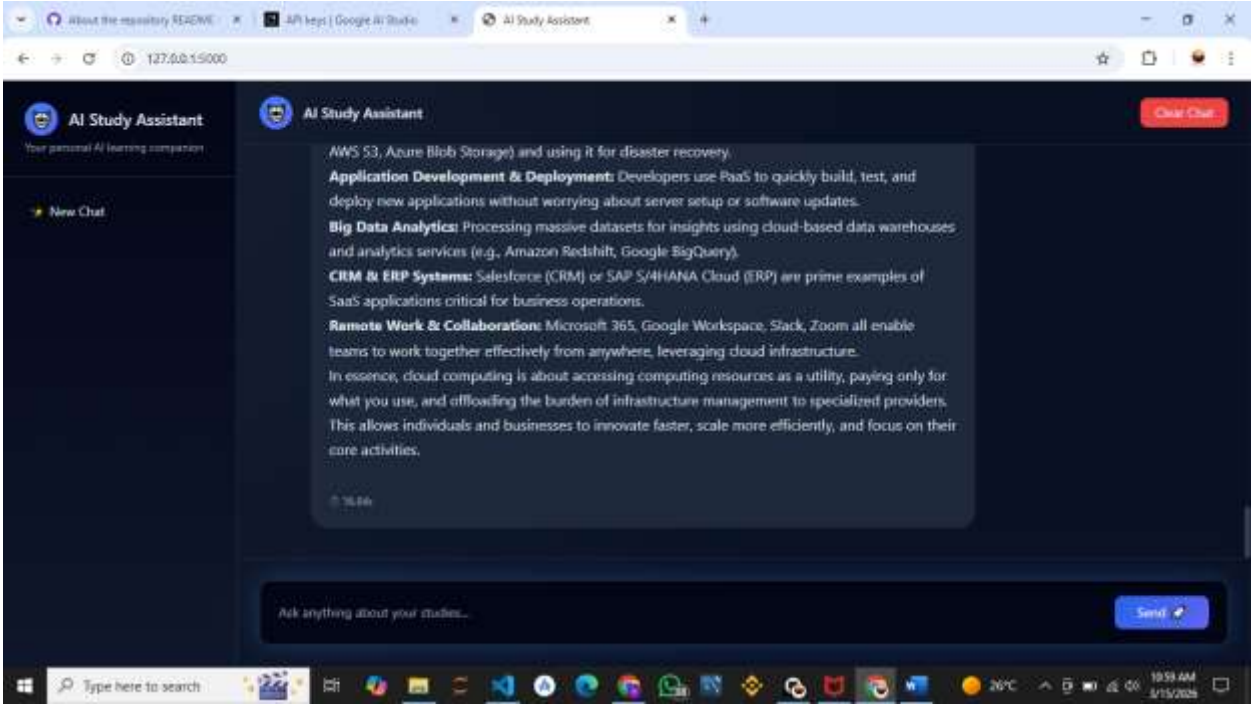

About the repository README
API keys | Google AI Studio
AI Study Assistant
127.0.0.1:5000
AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
Types of AI:
Narrow AI (or Weak AI): This is the AI we encounter today. It's designed and trained for a specific task (e.g., facial recognition, voice assistants, recommendation engines, self-driving cars *for the task of driving*). It can perform its specific task very well, sometimes even better than humans, but lacks general intelligence or consciousness.
General AI (or Strong AI/AGI - Artificial General Intelligence): This is hypothetical AI that would possess human-level cognitive abilities across a wide range of tasks, capable of understanding, learning, and applying intelligence to any intellectual task that a human can. It does not exist yet.
Superintelligence: A hypothetical AI that would surpass human intelligence across virtually all fields, including scientific creativity, general wisdom, and social skills.
Common Applications of AI:
Virtual Assistants: Siri, Alexa, Google Assistant
Recommendation Systems: Netflix, Amazon, Spotify
Self-Driving Cars: Tesla Autopilot, Waymo
Medical Diagnosis: Analyzing images for diseases, predicting patient outcomes
Ask anything about your studies...
Send
Type here to search


AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
Understand Language: Process and generate human language.
Adapt: Adjust to new situations and make decisions with some degree of autonomy.
How Does AI Work? AI systems are built using complex algorithms and models, often trained on vast amounts of data. Key techniques include:
Machine Learning (ML): A subset of AI where systems learn from data without being explicitly programmed. Instead of following pre-set rules, they identify patterns and make predictions or decisions.
Deep Learning (DL): A subfield of ML that uses artificial neural networks with multiple layers (hence "deep") to learn complex patterns, often excelling in tasks like image recognition, speech recognition, and natural language processing.
Natural Language Processing (NLP): Enables computers to understand, interpret, and generate human language.
Computer Vision: Allows machines to "see" and interpret visual information from the world (images, videos).
Robotics: Involves the design, construction, operation, and use of robots, often incorporating AI for perception, navigation, and decision-making.
Ask anything about your studies...
Send
Type here to search


AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
What is Artificial Intelligence?
Artificial Intelligence (AI) is a broad field of computer science focused on creating machines that can perform tasks that typically require human intelligence.
In simpler terms, AI aims to make computers "think" and "learn" like humans do, enabling them to understand, reason, perceive, and act in ways that mimic human cognitive abilities.
Here's a breakdown of what that entails:
What is "Intelligence" in AI? It refers to the ability of a machine to:
Learn: Acquire information and rules for using the information (e.g., through machine learning).
Reason: Use rules to reach approximate or definite conclusions.
Problem-solve: Develop strategies to solve specific challenges.
Perceive: Understand and interpret sensory input (like images, sounds, or text).
Understand Language: Process and generate human language.
Ask anything about your studies...
Send
Type here to search


AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
What is Artificial Intelligence?
Ask anything about your studies...
Send
Type here to search


AI Study Assistant
Your personal AI learning companion
New Chat
AI Study Assistant
Clear Chat
Ask anything about your studies...
Send
Type here to search



Final Code’s

Front-end

```
<!DOCTYPE html>
<html lang="en">

<head>

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>AI Study Assistant</title>

<script src="https://cdn.tailwindcss.com"></script>
<script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
<script
src="https://cdn.jsdelivr.net/npm/highlight.js/lib/common.min.js"></script>

<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/highlight.js/styles/github-dark.min.css">

<style>
```

```
body{
background:
radial-gradient(circle at 10% 10%,#1e293b,#020617 60%);
font-family:Inter,system-ui,-apple-system,sans-serif;
color:#e2e8f0;
overflow:hidden;
}

/* glassmorphism */

.glass{
background:rgba(15,23,42,.55);
backdrop-filter:blur(30px);
border:1px solid rgba(255,255,255,.05);
}

/* glowing borders */

.glow{
box-shadow:
0 0 20px rgba(59,130,246,.25),
0 0 60px rgba(59,130,246,.12);
}

/* sidebar */

.sidebar{
background:
linear-gradient(180deg,#020617,#0f172a);
border-right:1px solid rgba(255,255,255,.05);
}

/* bubbles */

.bubble{
max-width:70%;
padding:18px 20px;
border-radius:18px;
line-height:1.7;
position:relative;
}

.user{
background:linear-gradient(135deg,#2563eb,#3b82f6);
color:white;
```

```
}

.ai{
background:#1e293b;
}

/* message animation */

.message{
animation:fade .3s ease;
}

@keyframes fade{
from{opacity:0;transform:translateY(8px)}
to{opacity:1;transform:translateY(0)}
}

/* avatars */

.avatar{
width:36px;
height:36px;
border-radius:50%;
display:flex;
align-items:center;
justify-content:center;
background:linear-gradient(135deg,#3b82f6,#6366f1);
font-size:18px;
}

/* typing animation */

.typing span{
width:6px;
height:6px;
background:#94a3b8;
display:inline-block;
border-radius:50%;
margin:3px;
animation:blink 1.4s infinite both;
}

.typing span:nth-child(2){animation-delay:.2s}
.typing span:nth-child(3){animation-delay:.4s}
```

```
@keyframes blink{
0%,80%,100%{opacity:0}
40%{opacity:1}
}

/* input */

.inputbox{
background:#020617;
border:1px solid rgba(255,255,255,.1);
}

/* send button */

.sendbtn{
background:linear-gradient(135deg,#2563eb,#6366f1);
transition:.25s;
}

.sendbtn:hover{
transform:scale(1.05);
box-shadow:0 0 25px rgba(99,102,241,.5);
}

/* copy tool */

.tools{
position:absolute;
top:6px;
right:8px;
opacity:0;
font-size:12px;
cursor:pointer;
}

.bubble:hover .tools{
opacity:1;
}

/* scrollbar */

::-webkit-scrollbar{width:6px}
::-webkit-scrollbar-thumb{background:#334155;border-radius:10px}

/* code blocks */
```

```
pre{
border-radius:12px;
padding:14px;
overflow:auto;
}

</style>

</head>

<body class="h-screen flex">

<!-- SIDEBAR -->

<div class="w-64 sidebar hidden md:flex flex-col">

<div class="p-6 border-b border-gray-800">

<div class="flex items-center gap-3 text-lg font-semibold">

<div class="avatar">🤖</div>

AI Study Assistant

</div>

<div class="text-xs text-gray-400 mt-1">

Your personal AI learning companion

</div>

</div>

<div class="flex-1 p-4 space-y-2 text-sm">

<div class="p-3 rounded-lg hover:bg-gray-800 cursor-pointer transition">

✨ New Chat

</div>

</div>
```

```
</div>

<!-- MAIN -->

<div class="flex-1 flex flex-col glass">

<!-- HEADER -->

<div class="flex justify-between items-center px-6 py-4 border-b border-gray-
800">

<div class="flex items-center gap-3">

<div class="avatar">🤘</div>

<div class="font-semibold">

AI Study Assistant

</div>

</div>

<button
id="clearBtn"
class="bg-red-500 px-4 py-1.5 text-sm rounded-lg hover:bg-red-600 transition">

Clear Chat

</button>

</div>

<!-- CHAT -->

<div
id="chat"
class="flex-1 overflow-y-auto p-8 space-y-6">

</div>

<!-- INPUT -->

<div class="p-6 border-t border-gray-800">
```

```
<div class="inputbox glow rounded-xl flex items-center gap-4 px-5 py-4">

<textarea
id="question"
rows="1"
placeholder="Ask anything about your studies..."
class="flex-1 bg-transparent outline-none resize-none text-sm"></textarea>

<button
id="sendBtn"
class="sendbtn px-6 py-2 rounded-lg text-sm font-semibold">

Send 🚀

</button>

</div>

</div>

</div>

<script>

const chat=document.getElementById("chat")
const input=document.getElementById("question")
const sendBtn=document.getElementById("sendBtn")
const clearBtn=document.getElementById("clearBtn")

function scrollBottom(){
chat.scrollTop=chat.scrollHeight
}

function copyText(text){
navigator.clipboard.writeText(text)
}

function addMessage(text,sender,typing=false){

const msg=document.createElement("div")
msg.className=`message flex ${sender=="user"?"justify-end":""}`

if(typing){

msg.innerHTML=`
```

```
<div class="flex gap-3">

<div class="avatar">🤖</div>

<div>

<div class="text-xs text-gray-400 mb-1">
AI thinking...
</div>

<div class="bubble ai">

<div class="typing">
<span></span><span></span><span></span>
</div>

</div>

</div>

</div>
`

}else{

let bubble=sender=="user"?"user":"ai"
let content=sender=="ai"?marked.parse(text):text

msg.innerHTML=`

<div class="bubble ${bubble}">

<div class="tools"
onclick="copyText(\`${text}\`)">📋</div>

${content}

</div>
`

}

chat.appendChild(msg)
```

```
document.querySelectorAll("pre code").forEach(block=>{
hljs.highlightElement(block)
})

scrollBottom()

return msg
}

async function sendQuestion(){

const question=input.value.trim()
if(!question)return

addMessage(question,"user")

input.value=""

const typingMsg=addMessage("","ai",true)

sendBtn.disabled=true

try{

const res=await fetch("/ask",{
method:"POST",
headers:{"Content-Type":"application/json"},
body:JSON.stringify({question})
})

const data=await res.json()

typingMsg.innerHTML=`

<div class="flex gap-3">

<div class="avatar">🤖</div>

<div class="bubble ai">

<div class="tools"
onclick="copyText(\`${data.answer}\`)">📋</div>

${marked.parse(data.answer)}
```

```
<br>

<span class="text-xs text-gray-400">
⏱ ${data.time}s
</span>

</div>

</div>
`

}
catch(e){

typingMsg.innerHTML=`<div class="bubble bg-red-600">Server Error</div>`

}

sendBtn.disabled=false

}

sendBtn.onclick=sendQuestion

input.addEventListener("keydown",(e)=>{
if(e.key==="Enter"&&!e.shiftKey){
e.preventDefault()
sendQuestion()
}
})

clearBtn.onclick=()=>{
chat.innerHTML=""
}

input.addEventListener("input",()=>{
input.style.height="auto"
input.style.height=input.scrollHeight+"px"
})

input.focus()

</script>

</body>
```

```
</html>
```

Backend

```python
from flask import Flask, request, jsonify, render_template
from google import genai
import time
import os
import logging

app = Flask(__name__)

# Logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)

# API Key (env variable supported)
API_KEY = os.getenv("GEMINI_API_KEY", "AIzaSyCbHRE0g_5gcL0732QS0ENXwSqhYRYRpsU")

# AI client
client = genai.Client(api_key=API_KEY)


# AI function (LOGIC SAME)
def ask_ai(question):
    try:
        response = client.models.generate_content(
            model="gemini-2.5-flash",
            contents=[question]
        )

        if hasattr(response, "text") and response.text:
            return response.text.strip()

        return "AI returned empty response."

    except Exception as e:
        logging.error(f"AI Error: {e}")
        return f"AI Error: {str(e)}"
```

```python
@app.route("/")
def home():
    return render_template("index.html")


# ASK API (LOGIC SAME)
@app.route("/ask", methods=["POST"])
def ask():
    try:
        data = request.get_json()

        if not data:
            return jsonify({"answer": "No data received", "time": "0.00"})

        question = data.get("question", "")

        if question == "":
            return jsonify({"answer": "Please enter a question", "time": "0.00"})

        start = time.perf_counter()

        answer = ask_ai(question)

        end = time.perf_counter()

        return jsonify({
            "answer": answer,
            "time": f"{end-start:.2f}"
        })

    except Exception as e:
        logging.error(e)
        return jsonify({
            "answer": f"Server error: {str(e)}",
            "time": "0.00"
        })


if __name__ == "__main__":
    app.run(debug=True)
```